Portfolio Health Monitor

TechnicalWriteup —TogetherFundInternshipAssignment
Kushagra | kushagrabhargava93@gmail.com | July2026

---

1. Problem

TogetherFund's partners typically learn a portfolio company is in trouble when the founder calls — layoffs already announced, runway at three weeks. With 36+ companies in the portfolio, no one person can track all of them manually every week. There are no proactive signals. By the time distress surfaces, the window for meaningful help has usually already closed.

The Portfolio Health Monitor is built to change that. It scans public signals weekly for every company — headcount trends, news mentions, job postings — and uses an LLM to interpret the combination into a red/yellow/green health status with actionable reasoning. Partners open the dashboard on Monday morning and immediately know which companies need a call.

2. What It Does

The monitorruns a 4-step signal collection loop per company, then passes everything to Groq/Llama for holistic interpretation. The key design decision is that no single signal is treated as conclusive — a headcount drop could be a layoff or a restructuring. The LLM reads all signals together and reasons about the combination.

Signal collection pipeline:

- **Step 1 — Headcount delta:** 6-month history from seed data. Computes delta percentage and trend direction.
- **Step 2 — News scan:** DuckDuckGo search for company mentions in the last 30 days. Absence of press is itself a signal.
- **Step 3 — Job postings:** Searches LinkedIn, Greenhouse, and Lever. If the narrow search returns fewer than 2 results, the agent automatically fires a broader fallback query before concluding.
- **Step 4 — GitHub activity:** Proxy commit score for open-source companies. Skipped cleanly if no GitHub handle is on file.

After collection, a single structured LLM call outputs a **HealthSummary**: status, headline, 2–4 sentences of reasoning citing specific signals, a recommended partner action, and a confidence level. Everything is persisted to SQLite so the dashboard can show week-over-week trend history, not just today's snapshot.

3. Architecture

Stack:

- **Python 3.11 + Streamlit** — single-file dashboard, no separate frontend build step
- **Groq API (llama-3.3-70b-versatile)** — free tier, fast inference, OpenAI-compatible client
- **duckduckgo-search 5.3.1b1** — pinned deliberately; v6+ pulls in pyreqwest-impersonate which requires Rust/Cargo to compile on Windows and breaks on pip install
- **SQLite** — one row per company per scan; enables delta detection across weeks with zero infrastructure
- **APScheduler** — background cron fires every Monday 07:00; also triggerable on-demand from the dashboard
- **Pydantic v2** — validates LLM JSON output at parse time; malformed fields raise errors rather than passing through silently

File structure:

```
dashboard.py — Streamlit UI: cards, live log, charts, drill-down
app/monitor.py — Agent loop: collect signals → Groq synthesis → persist
app/database.py — SQLite read/write; deduplication; history queries
app/models.py — Pydantic schemas: SignalSnapshot, HealthSummary
app/scheduler.py — APScheduler cron + on-demand trigger
data/companies.json — Seed: 10 Together Fund portfolio companies
```

4. Why No LangChain / LangGraph

An agentic loop needs three things: a loop, a decision about what to do next, and a tool call. All three are implemented directly in roughly 40 lines of **monitor.py**. LangGraph wraps these same primitives in a graph abstraction that adds overhead, obscures the reasoning surface, and signals to a technical reviewer that the engineer reached for a framework rather than understanding the primitives. For a focused 4-step signal collection loop, the scratch-built version is the correct choice.

5. Key Technical Decisions

— **duckduckgo-search pinned to 5.3.1b1:** v6 introduced pyreqwest-impersonate as a dependency, which requires Rust/Cargo compilation. On Windows with Anaconda this fails at the wheel build stage. Pinning to the last pre-Rust version eliminates the issue entirely.

— **6-hour cache guard:** the agent checks whether fresh data already exists before hitting DuckDuckGo. This prevents rate-limit failures mid-demo and makes repeated refreshes idempotent.

— **Groq over Anthropic:** Groq's free tier with llama-3.3-70b-versatile is fast enough for a live demo and has no cost barrier. The client is OpenAI-compatible — swapping back to Claude requires changing one line.

— **Streamlit over React:** eliminates a separate frontend build step, SSE streaming setup, and npm dependency management. The live log console and Plotly charts cover everything needed for a compelling demo.

— **Graceful fallback on LLM failure:** if the Groq call fails or returns malformed JSON, the agent returns a yellow status with the error as reasoning. The dashboard never crashes mid-demo.

6. Limitations & Production Path

The current version uses seeded synthetic headcount data because LinkedIn blocks scraping at scale. In production this would be replaced by Proxycurl or founder-reported quarterly updates. News and job signals are live DuckDuckGo searches today; a Perplexity or SerpAPI integration would improve coverage meaningfully.

The dashboard currently requires a partner to open it. The highest-priority production addition is a Slack webhook so red-status companies trigger an automatic alert rather than waiting for a Monday login. Beyond that: PostgreSQL for multi-partner concurrent access, and expansion from 10 seed companies to all 59 Together Fund portfolio companies — which requires no code changes, only additions to companies.json.

The core insight: any script can pull a headcount number. What this does is read headcount + news + hiring together and surface: 'Frontrow lost 13% of its team in 60 days with zero press and zero new job postings. Call the founder this week.' That's what a partner actually needs on a Monday morning.